

A Multi-Stage Predictive Maintenance Framework for Railway Engines Using Machine Learning and Deep Learning^{*}

Nusaybah H. Altrabolsi^a, Fay M. Redha^a, Rahaf A. Alamri^a and Dana A. Alghamdi^a

^aCollege of Engineering and Computer Science, University of Jeddah, Jeddah, Saudi Arabia

ABSTRACT

Reliable railway operations depend on the continuous performance of critical components, where unexpected failures in engines and sensors can cause severe service disruptions and escalate maintenance costs. Traditional maintenance strategies, whether reactive or scheduled, often fail to detect early indicators of failure, leading to unplanned downtime and inefficiencies.

This paper presents a multi-stage predictive maintenance (PdM) framework that integrates machine learning and deep learning models to anticipate failures in railway systems. Using real-world sensor data from the Saudi Railway Company (SAR), the framework processes time-series data to classify failure types and detect early degradation, thereby enhancing operational reliability. The system was evaluated using Random Forest, XGBoost, and deep sequence models (LSTM, GRU), optimized for time-series analysis. The experimental results showed improved results in failure detection and early degradation prediction, highlighting the potential of the framework to reduce unplanned downtime and optimize maintenance scheduling.

1. Introduction

Maintenance is a key element in ensuring the long-term reliability and operational effectiveness of machinery and infrastructure, particularly in sectors such as logistics and transportation where consistent performance is essential. With increased reliance on complex mechanical systems, like engines, hydraulic units, and transportation fleets, including trains, unplanned equipment failures can lead to serious operational disruptions, service interruptions, and elevated maintenance costs. In a predictive maintenance focused organization, it is important to distinguish between the main maintenance strategies to understand their roles and limitations, and how predictive methods offer practical improvements. Accordingly, the differences between the various maintenance strategies will be clarified, including:

1. **Reactive Maintenance:** Performed only after a failure occurs. While sometimes unavoidable, continual reliance on reactive work leads to unnecessary unplanned downtime, emergency repairs, and higher costs.
2. **Preventive Maintenance:** Scheduled at fixed intervals regardless of equipment condition. This reduces the chance of failure but may result in unnecessary maintenance or missed problems between checks.
3. **Time-/Rule-Based Maintenance:** Based on predefined operating hours or usage cycles, assuming consistent wear across assets. This may not always reflect actual equipment usage.
4. **Predictive Maintenance:** Monitors equipment condition and performance in real time to anticipate failures before they occur, improving reliability and reducing waste.

As described by [1], predictive maintenance systems are commonly divided into four types. Data-driven systems rely on recorded sensor data to identify how the system behaves and use those patterns to anticipate failures. Statistical systems use tools like trend analysis and control charts to find signs of abnormal performance. Physics-based systems apply engineering models to represent how components wear out, helping to explain failure based on physical conditions. Knowledge-based systems depend on expert rules and experience to evaluate system status and decide when maintenance is needed. These maintenance strategies are summarized and visualized in the classification chart shown in Figure 1, illustrating the transition from reactive to predictive and prescriptive maintenance approaches.

In this study, we focus on leveraging data-driven predictive maintenance, which is especially effective in dynamic environments where system performance changes over time. By using sensor data from critical components, we can detect early signs of wear or failure, allowing for timely preemptive maintenance that minimizes service disruptions and enhances system reliability.

This work collaborates with the Saudi Railway Company (SAR) and utilizes real-world data from their railway infrastructure. The aim is to develop a multi-stage predictive maintenance framework that utilizes machine learning (ML) and deep learning (DL) models for failure detection and classification, thereby improving the operational reliability of the railway system.

The rest of the paper is structured as follows: Section 2 reviews related work; Section 3 describes the methodology; Section 4 presents experimental results; and Section 5 concludes the study.

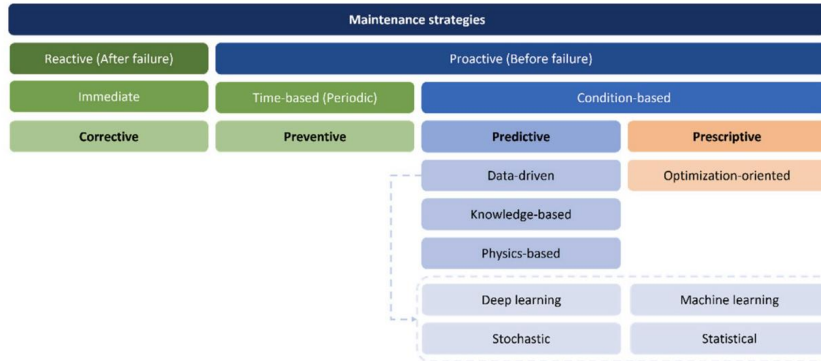


Figure 1: Classification of maintenance strategies: from reactive to predictive and prescriptive (adapted from Davari et al., 2021).

2. Related Work

2.1. Applications of Predictive Maintenance in Railway Systems

In recent years, the majority of railway companies have transitioned from conventional manual maintenance practices to data-driven predictive maintenance systems. This shift has been motivated by both business performance goals and advances in technology. Key business drivers for adopting predictive maintenance include reducing unplanned downtime, lowering maintenance costs, and improving overall process efficiency. Initially, fault detection practices relied heavily on manual documentation and strict operational standards. For instance, [2] describes how metro train operators traditionally depended on manually recorded information to identify faults. Although effective to an extent, these methods struggled to anticipate unusual failures or respond to evolving system behavior. The gradual adoption of machine learning (ML) and deep learning (DL) techniques introduced more scalable and responsive maintenance strategies. In [3], MLP (multi-layer perceptron) models were applied to monitor pump systems within a pulp mill environment, a solution that was later adapted to railway systems. Methods such as sensor fusion and sliding window approaches have since been employed for real-time fault prediction, enabling continuous monitoring of railway assets. Further applications have demonstrated the effectiveness of integrating sensor data with predictive models. For example, [4] discusses how the use of edge computing combined with vibration and temperature sensor readings allowed operators to detect faults in motors and bearings with high accuracy, minimizing unexpected breakdowns. Predictive maintenance has also influenced strategic planning in railway operations. High-speed rail networks, as described in [5], have incorporated edge devices and onboard optical cameras to monitor rail infrastructure conditions in real-time. The availability of real-time monitoring data has significantly improved the precision of maintenance scheduling and enhanced system reliability. These examples illustrate the railway sector's incremental transition from labor-intensive manual maintenance toward intelligent, automated facilities. By aligning technological advancements with corporate objectives, railway companies are not only achieving greater operational efficiency but also enhancing safety and extending the lifespan of critical assets.

2.2. Preprocessing Techniques

Handling the data is a critical part of predictive maintenance to pre-process the raw sensor data for the machine learning models. It entails numerous preprocessing operations such as the removal of missing values, outliers, and duplicates, and normalization of the data. Sensor data in industrial and rail systems tends to contain noise and uneven sampling, and this can reduce the accuracy of the model. One of the standard procedures is to remove outliers and duplicate measures, then apply sliding window normalization, where a window of fixed width slides over the data to achieve consistent statistical properties from continuous measurement [3]. This helps to normalize the data input, which improves the learning of predictive models. Also, resampling of the sensor data at fixed intervals (e.g., every 60 seconds) gives equidistant time spacing, and spike filtering removes unrealistically big jumps caused by sensor noise rather than actual system behavior. Removing irrelevant or biased features can make models stronger and easier to interpret [6].

Feature extraction is an important step in transforming raw data before feeding it into machine learning algorithms. It focuses on pulling out and calculating key statistical features that show important patterns and trends in the data. **Timestamp alignment** is also a data transformation process and is used for matching data readings with the timing of failure instances observed. It is particularly important in the creation of well-labelled datasets, which are essential for supervised learning [7].

For the additional enhancement of data representation, **autoencoders** are used. Models take high-dimensional data and convert it into a lower-dimensional set of features that preserve the most crucial information, making it possible to reconstruct the original data effectively. **Stacked autoencoders**, built from multiple layers of encoders and decoders arranged hierarchically, are frequently used when working with more complicated data structures. They assist in revealing hidden patterns within the dataset in addition to reducing its dimensionality [8]. When dealing with problems that have few failure samples or imbalanced class distributions, generating synthetic or simulated data becomes crucial. One approach is to create synthetic datasets through simulation techniques or by using generative models like **GANs**, helping to address the shortage of real-world failure examples. Moreover, **digital twins** are used for the simulation of complex industrial scenarios, generating diverse and high-quality training data. Simulated data has been helpful in filling gaps in datasets and the readiness of models for rare failure states.

Subsequently, undersampling techniques are applied to balance the dataset by reducing the majority of normal operation data so that the model is exposed to both normal and failure scenarios during training [7]. Industrial datasets can also be afflicted with other issues, such as a few labels, varying operational conditions, and needs for expert domain knowledge to effectively interpret the data.

2.3. Machine Learning Techniques in Predictive Maintenance

Several studies have shown that machine learning (ML) plays an important role in predictive maintenance (PdM), enabling the analysis of sensor data to identify early indicators of equipment failure. In both industrial and railway systems, ML facilitates intelligent maintenance planning by classifying faults and identifying potential failures before they occur [9]. When labeled fault data is available, supervised learning techniques are frequently used to train predictive models. Common algorithms include Artificial Neural Networks (ANN), Random Forests (RF), and eXtreme Gradient Boosting (XGBoost). Unlike some traditional classifiers, XGBoost is less sensitive to class imbalance, making it suitable for detecting rare fault events. In predictive maintenance, XGBoost has been used to classify different types of fault and detect failures by learning from structured sensor data, and it has proven effective when used alongside other models [4]. For instance, ANN and ensemble models have been applied in metro train systems to spot issues like air and oil leaks, with some solutions built into real-time dashboards to help maintenance teams act quickly and avoid further problems [7]. Another study trained ANN models on time-series sensor data and demonstrated their ability to catch early warning signs of equipment failure [4]. An industrial application developed a predictive system using Multilayer Perceptrons (MLPs), trained on pump sensor data from a pulp processing plant. This system combined short-term statistical methods with long-term neural network forecasts, using sliding windows and features like standard deviation and power spectral density to enhance prediction performance in a live operational environment [3]. In contrast, unsupervised learning techniques are commonly adopted when labeled data is limited or unavailable. Techniques like autoencoders, clustering algorithms, and reconstruction-based anomaly detection are often used to understand normal equipment behavior and spot unusual patterns that may signal a problem. These methods are especially useful in early deployments or in high-risk environments, where actual failures are uncommon but can have serious consequences.. One example applied these methods to metro system compressors, successfully identifying anomalies without relying on predefined failure labels [7]. While machine learning has played an important role in predictive maintenance, there

are still some clear gaps that make it harder to apply in real-world settings. One major issue is the lack of failure data, which makes it difficult for models to learn from rare but critical events [4]. Another challenge is understanding how system behavior changes over time, which can be difficult to capture with traditional models [4]. Recent studies show that deep learning offers better solutions for these challenges, particularly in large and dynamic environments [7] and [10]. The next section takes a closer look at this approach.

2.4. Deep Learning Techniques in Predictive Maintenance

Deep learning plays a key role in predictive maintenance due to its ability to identify complex patterns in raw sensor data without extensive manual preprocessing [10]. Unlike traditional models, deep learning architectures consist of multiple layers that allow them to handle the variability and complexity often found in industrial operating conditions [5]. Amid the accelerating industrial transformation driven by the Fourth Industrial Revolution, deep learning has shown it can help predict failures better, improve operations, and make systems safer and more reliable, even though it had problems before, like the vanishing gradient, which new technology has mostly solved [10].

Recent studies have shown the importance of deep sequence models in improving predictive maintenance systems that use time-series sensor data [10]. These models are a deep learning approach designed to handle sequential information by identifying patterns across time steps. Recurrent Neural Networks (RNNs) and their variants, Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), are used in this context to detect changes in system performance. For example, [11] applied LSTM and GRU models to industrial sensor data and predicted faults up to one month before, with error rates below 10% energies.

Recurrent Neural Networks (RNNs) are used in predictive maintenance to analyze time-series sensor data and identify patterns that may signal early signs of equipment wear. They work by connecting information across time steps, making them useful for tracking changes in system performance. However, RNNs can have difficulty capturing long-term trends, particularly in complex or noisy environments [10].

Long Short-Term Memory (LSTM) networks are an improved type of RNN that can retain information over longer periods [10]. This makes them more effective at detecting gradual faults in systems that degrade over time. In railway applications, LSTMs have been used to monitor components such as traction cooling systems by learning from historical data, helping to prevent unexpected failures[6].

Gated Recurrent Units (GRUs) provide a simpler alternative to LSTMs while maintaining the ability to learn from sequential data. Because they use fewer parameters, GRUs are more efficient and better suited for real-time monitoring tasks that require fast processing with limited resources [10]. They have shown strong results in predicting faults in rail systems and are often used in edge-based deployments.

However, challenges remain. Engineers must carefully select deep learning architectures based on data types and operational constraints. Additionally, the availability of high-quality labeled datasets and the need for interpretability are ongoing limitations. Future directions involve leveraging techniques such as transfer learning and advanced data preparation to enhance model generalizability and transparency [10].

3. Methodology

This section outlines the methodology adopted for developing a multi-stage predictive maintenance (PdM) system. The pipeline processes raw time-series sensor data to detect, classify, and anticipate engine failures. It begins with data preprocessing, followed by three sequential prediction stages. Each stage is designed for a specific maintenance objective and combines machine learning and deep learning models. An overview of the system is shown in Figure 2, following the general structure proposed in [10] and validated by multi-sector approaches like [1].

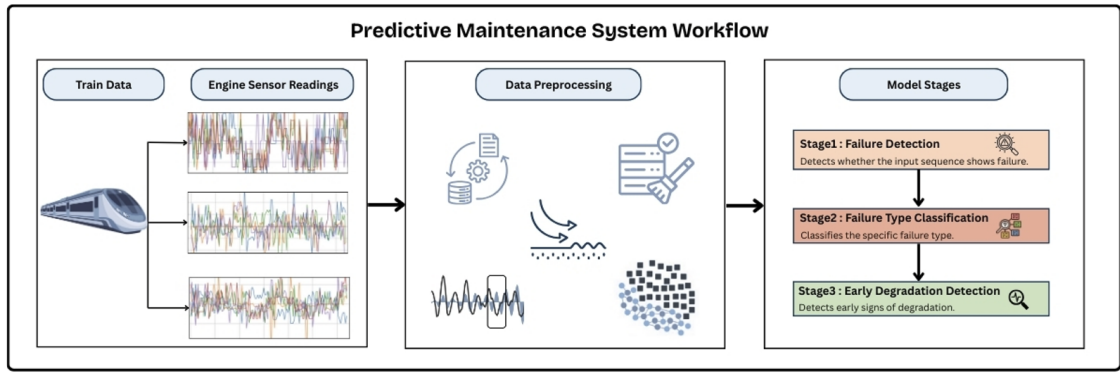


Figure 2: Predictive Maintenance System Workflow

3.1. System Architecture and Workflow

The predictive maintenance workflow is organized into three primary stages:

1. **Failure Detection (Stage 1):** Detect whether an input sequence indicates a failure or normal operation using binary classification.
2. **Failure Classification (Stage 2):** Classify the specific failure type for input sequences identified as failures, using multi-class classification.
3. **Early Degradation Detection (Stage 3):** Detect early signs of performance degradation in input sequences classified as normal, to anticipate potential future failures.

Each stage uses a sliding window technique to segment continuous time-series data into shorter sequences, as emphasized in [12] and [6]. Stages 1 and 2 apply a fixed window size and stride to create overlapping segments of equal length, consistent with best practices noted in [13]. For Stage 3, variable-length windows are used to better capture long-term degradation trends, as highlighted in [4]. All segments are normalized and filtered to ensure consistent data quality a critical preprocessing step also stressed in [7].

This structured workflow supports maintenance strategies, enabling detection of current failures, classification of fault types, and continuous system health monitoring, as outlined in [14].

3.2. Model Design

Each predictive stage is treated as a distinct machine learning task: binary classification in Stage 1, multi-class classification in Stage 2, and regression in Stage 3, aligning the model type with the nature of the required prediction.

3.2.1. Stage 1: Failure Detection (Binary Classification)

This stage determines whether the engine is functioning normally or exhibiting signs of failure, modeled as a binary classification task with labels “0” (No Failure) and “1” (Failure). Based on [4], the sliding window approach improves the representation of sequential sensor readings. Due to the class imbalance between typical and failure cases, oversampling is applied during training, following strategies in [4] and [7].

Machine Learning Models Two machine learning algorithms were evaluated:

- **Random Forest:** Configured with 100 trees and balanced class weights. This model demonstrated strong performance in industrial PdM systems [1].
- **XGBoost:** Implemented with 100 trees, a learning rate of 0.3, and a maximum depth of 6 ([8] and [4]).

Deep Learning Models Recurrent neural networks were used to capture temporal dependencies:

- **LSTM (Long Short-Term Memory):** Configured with 64 hidden units and a 0.3 dropout rate, the model employed a stacked architecture of two LSTM layers interleaved with dropout and dense layers. It followed training protocols similar to those in [6], achieving over 99% accuracy in failure prediction.
- **GRU (Gated Recurrent Unit):** Configured with 64 and 32 units. The dropout rate was reduced to 0.2 to enhance computational efficiency, as noted in [10]. Both models incorporated time-aware validation and early stopping to prevent overfitting.

3.2.2. Stage 2: Failure Classification (Multi-Class Classification)

This stage classifies the failure types identified in Stage 1 into categories such as power assembly, injector, turbocharger, or SRS / TRS. This multi-class setup builds on railway PdM methodologies outlined in [14]. Preprocessing and segmentation mirror Stage 1 to maintain consistency.

Machine Learning Models

- **Random Forest:** Retained its 100-tree configuration and was extended for multi-class output using stratified sampling ([1]).
- **XGBoost:** The model was set up for multi-class classification to predict probabilities for each class, as in [4].

Deep Learning Models

- **LSTM:** Modified with a softmax output layer and trained using categorical cross-entropy loss, aligning with methods in [6].
- **GRU:** Followed the same approach. Both models maintained time-aware validation to ensure robustness [10].

3.2.3. Stage 3: Early Degradation Detection

Focuses on identifying early degradation indicators in sequences classified as “No Failure” in Stage 1. This phase is modeled as a regression task to capture subtle deviations from normal behavior—an approach supported by [4].

Sequential Modeling Approach To model degradation trends, LSTM and GRU were adapted for continuous output:

- **LSTM:** Configured with 64 hidden units, a 0.3 dropout rate in the first layer, and 32 units with 0.2 dropout in the second layer. It was trained using mean squared error (MSE) loss, following methods from [6].
- **GRU:** Used a similar setup and has proven effective in maritime anomaly detection tasks [8].

3.3. Dataset

This study utilizes a dataset provided by the Saudi Railway Company (SAR), containing operational data collected from locomotive engine subsystems. The dataset captures failure events and regular operating conditions, offering a foundation for building predictive maintenance models using data recorded under real-world service environments.

3.3.1. Dataset Collection and Integration

The dataset was initially organized into 31 Excel files, each representing a separate failure event linked to a specific engine unit. Data were gathered between 2024 and 2025, covering various operational scenarios experienced in actual field conditions. Each file documents sensor measurements and related metadata over the course of the engine's operation, leading up to the failure.

To facilitate analysis and preprocessing, all 31 files were merged into a single consolidated dataset, enabling unified handling of the sensor readings and precise calculation of the data volume. The final merged dataset consists of 4,818 records, providing a substantial basis for training and evaluating predictive maintenance models.

3.3.2. Dataset Characteristics

Each file consists of time-series recorded at irregular intervals, reflecting the natural dynamics of engine behavior. The main columns include *Time Stamp* to track the timing of each measurement, *Asset Name* to identify the engine unit, and *failure_type* to indicate the failure category associated with the recorded data. Additionally, the dataset contains a wide range of sensor readings that describe the operational state of the engine over time.

The key components of each record are summarized in Table 1. Furthermore, the dataset captures four distinct types of failures, distributed as shown in Table 2.

Table 1: Key dataset columns and their descriptions

Column	Description
Time Stamp	Timestamp indicating when each sensor measurement was captured.
Asset Name	Identifier of the engine unit associated with the dataset.
failure_type	Label representing the failure category recorded for the engine event.
Sensor Columns	Time-dependent numeric features reflecting engine operational measurements.

Table 2: Failure event distribution across the dataset

Nr.	Failure Type	Subsystem	# Failures
1	Power Assembly	Engine	8
2	Injector	Engine	7
3	Turbocharger	Engine	9
4	SRS/TRS	Engine	7

These comprehensive data tables establish a solid understanding of the dataset's structure and distribution. This foundational overview sets the stage for the subsequent preprocessing steps, essential to prepare the data for modeling and analysis.

3.4. Data Preprocessing

Effective preprocessing is essential to transform raw sensor data into a clean, structured, and model-ready format. In predictive maintenance, time-series data collected from industrial sensors often suffer from missing values, noise, irregular sampling, and class imbalance, which can negatively impact model performance. This section outlines the comprehensive preprocessing adopted in this study, covering data extraction, cleaning, standardization, smoothing, and oversampling. Each step is designed to improve the data quality and ensure balanced class representation, thus improving the robustness and precision of the predictive maintenance models.

3.4.1. Data Extraction and Merging

The raw dataset consists of multiple Excel files containing timestamped sensor readings and failure event labels. To ensure consistency across the dataset, only the shared columns among all files were retained. The data was then merged into a single structure, with empty zero-filled rows inserted between files to delineate operational boundaries between different equipment units.[10], [7].

3.4.2. Data Cleaning

To improve data quality and reduce redundancy, irrelevant columns and duplicated features were removed. Additionally, highly correlated variables (Pearson correlation coefficient > 0.99) were dropped to reduce multicollinearity, which can degrade model interpretability and lead to overfitting. This step is crucial in high-dimensional time series data typical of industrial equipment, where sensor redundancy is common [4], [2].

3.4.3. Standardization and Smoothing

All numerical features were standardized using Z-score normalization to ensure that each feature contributed equally to model training, regardless of its original scale. A Hampel filter was applied to enhance data quality and detect and correct outliers based on local median deviation. This robust smoothing technique is particularly effective in preserving genuine temporal trends while mitigating the impact of anomalous sensor noise, recommended practice in predictive maintenance for rail and metro systems [6], [7].

3.4.4. Sliding Window Segmentation and Oversampling

A sliding window approach was applied to transform the time series into a format suitable for supervised learning. This method segmented the sequence into overlapping fixed-length windows, preserving temporal dependencies essential for learning progressive degradation and failure patterns [10], [7]. Due to the natural class imbalance, where failure events are sparse relative to normal operation, oversampling was performed to ensure sufficient representation of failure cases during model training. This strategy aligns with approaches in predictive maintenance literature aimed at mitigating skewed class distributions [4], [14].

4. Experimental Configurations

To comprehensively evaluate the PdM system, multiple experimental setups were tested across all stages:

- **Stage 1:**
 - Baseline models with no oversampling.
 - Models with oversampling applied to address class imbalance.
- **Stage 2:**
 - Models with oversampling to balance the multi-class dataset.
- **Stage 3:**
 - Models with oversampling,
 - Models with oversampling and hyperparameter tuning

This design clearly compares how data balancing and model optimization affect performance across binary, multi-class classification, and regression tasks.

5. Experimental Setup

All experiments were conducted on Google Colab within a CPU-based environment equipped with an Intel(R) Xeon(R) processor (2.3 GHz) and 16 GB of RAM (without GPU acceleration). The implementation was developed using Python (version 3.10), with TensorFlow (Keras) used for deep learning models, Scikit-learn for Random Forest, and XGBoost for gradient boosting. Pandas and NumPy were utilized for data processing, while Matplotlib and Seaborn handled visualization tasks.

The dataset, provided by the Saudi Railway Company (SAR), consisted of multiple Excel files containing time-series sensor readings and timestamps associated with engine failures. These files were merged into a unified dataset, retaining only the common columns to ensure consistency across all records. Preprocessing steps included handling missing values, removing duplicates, applying sliding window segmentation to generate overlapping sequences, and scaling features using StandardScaler and MinMaxScaler.

The dataset was divided into 80% training and 20% testing subsets, with oversampling applied to address class imbalance in failure detection and classification tasks. Machine learning and deep learning models were trained using standard configurations and best practices, including batch processing, adaptive optimizers, and early stopping to prevent overfitting. The Random Forest model was configured with 100 trees, while XGBoost was set with 100 trees, a learning rate of 0.3, a maximum depth of 6, and parallel processing enabled.

This experimental setup was designed in alignment with best practices reported in the predictive maintenance literature[4, 6].

6. Evaluation Metrics

We applied well-established evaluation metrics to evaluate the performance of the proposed predictive maintenance (PdM) system across all stages. These metrics were selected based on their proven effectiveness in predictive maintenance applications, ensuring a comprehensive assessment of classification and regression tasks. Each metric, along with its specific role within our PdM system, is defined below and supported by relevant references ([1]; [4]; [15]).

6.1. Stages 1 & 2: Classification Metrics

These stages involve binary and multi-class classification tasks. The following metrics were used:

Accuracy measures the proportion of correct predictions (including correctly identified failures and correctly identified normal cases) out of the total number of predictions. It is calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

In our PdM system, accuracy was used to assess the overall correctness of failure detection (Stage 1) and failure classification (Stage 2). This metric is widely adopted in PdM studies, such as the railway failure detection system described in [1] and supported by experiments in [4].

Precision quantifies the proportion of correctly predicted failures out of all cases that the model predicted as failures. It is calculated as:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

We applied precision to evaluate the system's ability to reduce false alarms, ensuring that detected failures are valid and actionable ([1], [9]).

Recall measures the proportion of actual failures correctly identified by the model. It reflects how effectively the system captures all real failures. The formula is:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

In our PdM system, recall was used to ensure that the model minimizes missed failures, which is critical to prevent unexpected breakdowns ([1],[12]).

F1-Score is the harmonic mean of precision and recall, balancing the trade-off between identifying failures accurately and avoiding false alarms. It is calculated as:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

We relied on the F1-score to provide a balanced metric that reflects the completeness and exactness of failure predictions, especially useful in our imbalanced dataset ([1],[16]).

ROC and AUC The Receiver Operating Characteristic (ROC) curve and the Area Under the Curve (AUC) are essential metrics for evaluating classification performance, particularly in imbalanced datasets. The ROC curve is a graphical representation that illustrates the diagnostic ability of a classifier by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings.

The formulas are defined as follows:

$$\text{TPR} = \frac{TP}{TP + FN} \quad \text{and} \quad \text{FPR} = \frac{FP}{FP + TN}.$$

The ROC curve helps visualize the trade-off between sensitivity (recall) and the false alarm rate, aiding in the selection of an optimal threshold that balances these factors. The AUC provides a single scalar value that summarizes the overall performance of the ROC curve. It represents the probability that a randomly chosen positive instance is ranked higher than a negative one. An AUC value of 1.0 indicates perfect classification, while 0.5 suggests no discriminative power.

In our PdM system, the ROC curve and AUC were applied in Stages 1 and 2 to assess and compare model performance across thresholds, ensuring a robust evaluation of the failure detection and classification models [4, 6].

6.2. Stage 3: Regression/Degradation Metrics

This stage involves regression and anomaly detection tasks. The following metrics were applied:

Mean Squared Error (MSE) MSE calculates the average of the squared differences between the predicted values and the actual sensor readings. It penalizes larger errors more heavily. The formula is:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (5)$$

We used MSE to measure how closely our degradation predictions matched actual sensor trends, ensuring the model's overall accuracy. This metric is widely used in PdM systems for evaluating degradation prediction models ([15];[6]).

Root Mean Squared Error (RMSE) In our PdM system, RMSE was applied to interpret prediction errors in real-world units, providing clearer insight into performance. The formula is:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (6)$$

This metric complements MSE by returning errors in the same unit as the original sensor values ([15], [7]).

Mean Absolute Error (MAE) We included MAE to provide a straightforward, interpretable measure of average prediction error for degradation estimation. The formula is:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (7)$$

MAE's simplicity and effectiveness are highlighted in PdM benchmarks ([15]; [17]).

7. Experimental Results

This section presents the experimental evaluation of the proposed predictive maintenance (PdM) system, covering the full pipeline:

- **Stage 1:** Failure detection (binary classification).
- **Stage 2:** Failure type classification (multi-class classification).
- **Stage 3:** Early degradation detection (regression).

The models' performance was assessed using standard evaluation metrics, as outlined in Section 6. The experiments were conducted across different configurations, including baseline models, oversampling for balancing classes, and hyperparameter tuning for deep learning models. In addition to standard evaluation metrics, we include ROC-AUC to provide a robust measure of classification performance, which is particularly useful in imbalanced settings.

7.1. Stage 1: Failure Detection Results

We evaluated four models: **Random Forest**, **XGBoost**, **LSTM**, and **GRU**. Two experimental setups were explored:

- **Baseline (No Oversampling):** Models were trained on the imbalanced dataset.
- **With Oversampling:** Synthetic samples were added to balance the failure and no-failure classes.

Key Findings:

- Deep learning models (LSTM and GRU) and machine learning models (Random Forest and XGBoost) initially showed poor performance due to class imbalance. However, after applying oversampling with sliding windows, all models improved significantly, achieving much better recall and F1-scores.

- The performance gap between deep learning and machine learning models became minimal after oversampling, indicating that oversampling played a key role in leveling the performance across different model types, as highlighted in prior PdM studies (e.g., [4], [1]).

Table 3: Stage 1: Failure Detection Performance Metrics

Model	Oversampling	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	No	0.99	0.5	0.5	0.5	0.5
Random Forest	Yes	1	1	1	1	1
XGBoost	No	0.99	0.5	0.5	0.5	0.5
XGBoost	Yes	1	1	1	1	1
LSTM	No	0.99	0	0	0	0.67
LSTM	Yes	0.99	0.98	1	0.99	1
GRU	No	0.99	0	0	0	0.71
GRU	Yes	0.99	0.98	0.99	0.98	1

Note: The results confirm the advantage of using deep sequence models for binary failure detection in time-series PdM applications [6].

7.2. Stage 2: Failure Type Classification Results

This involved multi-class classification of failure types, covering Power Assembly, Injector, Turbocharger, and SRS/TRS. All models were trained using oversampling to balance the classes.

- LSTM and GRU demonstrated the strongest overall performance, particularly in underrepresented classes (Injector and SRS/TRS).
- XGBoost outperformed Random Forest but showed some difficulty with less frequent classes, consistent with trends reported in [14, 6].

Table 4: Stage 2: Multi-Class Classification Metrics (Oversampling Setup)

Model	Precision	Recall	F1 Score	Accuracy	ROC-AUC
Random Forest	1	1	1	1	1
XGBoost	1	1	1	1	1
LSTM	0.99	0.99	0.99	0.99	1
GRU	0.99	0.99	0.99	0.99	1

Note: We include Macro ROC-AUC to evaluate multi-class discrimination performance. The ROC-AUC metric aggregates performance across all failure types, offering a holistic view of model capability [6]. Multi-class ROC curves are provided in Appendix B.

4.3.3 Stage 3: Early Degradation Detection Results

Focused on regression-based degradation prediction, aiming to detect subtle signs of early failure in sequences classified as normal in Stage 1. We tested LSTM and GRU only, as deep sequence models are more effective for continuous output tasks.

Two setups were performed:

- **(A) Oversampling Results:** Regression performance was measured using MSE, RMSE, and MAE.
- **(B) Hyperparameter Tuning:** Additional tuning to optimize predictive accuracy.

(A) *Oversampling Results* GRU achieved slightly lower error metrics than LSTM, confirming its strength in modeling gradual degradation trends. These outcomes support prior literature on GRU's efficiency in time-series regression ([8], [6]).

Table 5: Stage 3: Oversampling – Regression Error Metrics

Model	MSE	RMSE	MAE
LSTM	0.0812	0.2849	0.2452
GRU	0.0824	0.2870	0.2466

(B) *Hyperparameter Tuning Results* Fine-tuning of parameters (epochs, batch size, learning rate, dropout, and network architecture, e.g., Bidirectional layers) produced modest gains, further reducing error rates for both models.

Table 6: Stage 3: Hyperparameter Tuning Results for Regression

Model	Epochs	Batch Size	Learning Rate	Dropout	MSE	RMSE	MAE
LSTM	20	64	0.0008	-	0.0128	0.1130	0.0775
LSTM	50	64	0.001	(0.2-0.3)	0.0128	0.1130	0.0883
GRU	20	64	0.0008	-	0.0167	0.1291	0.1031
GRU	50	64	0.001	(0.2-0.3)	0.01	0.1002	0.0738

These findings emphasize the importance of tuning deep learning models for optimal performance in PdM regression tasks [6], [17]. *Note: ROC-AUC was not applicable in Stage 3 as it involved regression rather than classification.*

To train the regression models effectively, it is crucial to provide accurate degradation scores that represent the asset's health trajectory over time. The following subsection outlines the method used to generate these ground truth labels.

Ground Truth Generation for Degradation Modeling

In the absence of explicit degradation labels within the dataset, the ground truth for the degradation prediction task was synthetically generated based on the relative position of each time window within an engine's operational timeline. For each segmented window of sensor readings, a degradation score was assigned using the following linear scaling formula:

$$\text{Degradation Score} = \frac{i}{N - L} \quad (8)$$

where i denotes the starting index of the window, N is the total number of data points for the engine, and L is the window length.

This approach assumes that the health of the system degrades progressively over time, with the degradation score increasing from 0 (healthy) at the beginning of the sequence to 1 (near failure) at the end of the sequence. Although this method simplifies degradation to a linear trajectory, it serves as a practical proxy in scenarios where real-world degradation metrics are unavailable.

Such synthetic labeling strategies are commonly adopted in the predictive maintenance literature. For example, [18] and [19] discussed using synthetic labels in condition monitoring when true degradation data is missing. Similarly, [20] applied this method where degradation progression was inferred by normalizing the cycle index to represent health status. These works highlight that linear degradation modeling, though idealized, provides an effective baseline for training degradation prediction models when only end-of-life labels are available. .

Degradation Curve Evaluation

To provide deeper insight into the performance of the degradation prediction model, we conducted an experiment that visualizes the full degradation trajectory of a railway asset over its operational period. This experiment examines how the model interprets continuous sensor readings to track gradual wear and deterioration until failure.

Figure 3 illustrates the predicted degradation curve based on sensor data from the start of monitoring through to failure. The curve shows the asset's health transitioning from **Normal** (green) to **Maintenance Soon** (yellow) and finally to **Imminent Failure** (red), based on predefined degradation thresholds described in the methodology.

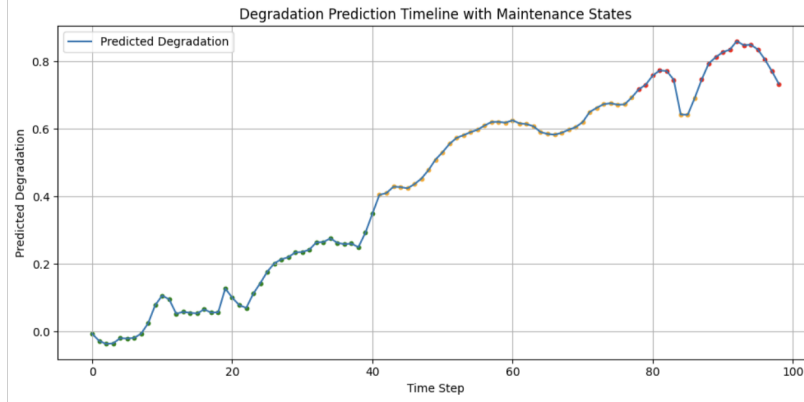


Figure 3: Predicted degradation timeline showing asset health progression over time.

The results confirm the model's ability to track long-term degradation effectively. Initially, the asset remains in a stable condition (green), but as degradation accumulates, it crosses into early warning (yellow) and eventually reaches the critical failure zone (red). This demonstrates the model's effectiveness in providing actionable insights for proactive maintenance scheduling.

Pipeline Execution: Sample Query Demonstration

To demonstrate the end-to-end functionality of the predictive maintenance pipeline, we tested a real query across all three stages of the system. This walkthrough highlights how the system processes a new sensor reading and dynamically determines the appropriate action based on the result at each stage Figure 4.

The pipeline is structured as follows:

- **Stage 1: Failure Detection.** The input query is first passed through a binary classification model to determine whether it indicates a failure.
- **Stage 2: Failure Type Classification.** If Stage 1 detects a failure, the system classifies the specific failure type using a multi-class classification model.
- **Stage 3: Degradation Prediction.** If no failure is detected in Stage 1, the system proceeds to predict the degradation score using a regression model to assess the asset's health over time.

results were obtained:

The following output provides a full trace of the pipeline's execution for this query:

1. **Stage 1:** The model output was "*No Failure*", indicating that the asset was operating normally at the time of the query.
2. **Stage 2:** Skipped, as Stage 1 confirmed no failure.
3. **Stage 3:** The system predicted a degradation score of approximately **0.38**, corresponding to a "**Normal**" maintenance state (based on thresholds defined as: Normal 0.4; Maintenance Soon: 0.4–0.7; Imminent Failure: > 0.7).

This demonstration confirms the pipeline's ability to dynamically route queries based on detected outcomes, ensuring comprehensive failure detection and continuous monitoring of asset health. It showcases the practical value of the multi-stage framework in handling real-world operational data.

Pipeline Final Result:									
{'stage_1_result': 'No Failure', 'stage_3_degradation_score': 0.3766689896583557, 'stage_3_maintenance_status': 'Normal'}									
Query sample used (Stage 1):									
	AmbTmP	APCClB	APImRbP	ATImRbF	AwTF	Bar Prs	Batt V	\	
3013	1.009824	1.071121	-0.356205	2.040333	1.448074	-0.839725	-1.223479		
	CA V	ChpDty	DCL1V	...	TmIj12	TmIj13	TmIj14	\	
3013	-1.253379	0.373748	-0.434597	...	-0.100347	0.043578	1.024168		
	TmIj15	TmIj16	TnSt	TPU RPM	WDTM1	WPEgILP	WPEgOTP		
3013	-0.070386	-0.274426	-0.300412	0.262998	-1.232397	-1.356615	-1.054705		

Figure 4: Pipeline output showing the query's processing through stages.

8. Discussion

This study introduces a multi-stage predictive maintenance (PdM) framework that integrates both machine learning and deep learning models to enhance the reliability and operational safety of railway assets. The experimental results, based on real-world data from the Saudi Railway Company (SAR), validate the framework's ability to detect failures, classify fault types, and predict long-term degradation, underscoring its practical relevance in industrial settings.

The findings from **Stage 1**, which focuses on binary failure detection, demonstrate that oversampling significantly improved the recall and F1-score of all models. Random Forest and XGBoost models, despite their solid baseline performance, showed marked improvements post-oversampling, consistent with prior research indicating the importance of balancing techniques for handling imbalanced datasets. Notably, deep sequence models such as LSTM and GRU exhibited superior performance, highlighting their capability to capture temporal dependencies in time-series sensor data. These results align with existing literature emphasizing the strength of recurrent architectures in sequential pattern recognition.

In Stage 2, tasked with multi-class failure type classification, deep learning models again outperformed traditional machine learning counterparts, particularly in minority classes such as Injector and SRS/TRS faults. While XGBoost maintained robust performance across major classes, its precision for rarer failure types was slightly lower, reaffirming challenges associated with class imbalance in multi-class setups. The inclusion of macro-averaged ROC-AUC and multi-class F1-scores provided a comprehensive view of each model's discriminative power, ensuring that the evaluation captured both global and class-specific performance trends.

Stage 3 focused on predicting degradation scores to enable proactive maintenance scheduling. Both LSTM and GRU models demonstrated a strong capacity to capture long-term degradation patterns, with GRU showing marginally better performance in terms of mean squared error (MSE), consistent with its design for computational efficiency in sequential data tasks. The degradation curve visualization offered additional insight, confirming the model's ability to track gradual wear from normal operation to imminent failure. This visualization serves as a critical tool for maintenance teams, providing a real-time perspective on asset health.

A unique contribution of this work is the full pipeline execution, which dynamically routes each query through the appropriate stages based on real-time outputs. The sample query demonstration confirmed the system's operational logic: accurately identifying normal operation at **Stage 1**, bypassing **Stage 2** when no failure was present, and delivering a precise degradation assessment at **Stage 3**. This flexible, modular design ensures that both reactive and proactive maintenance needs are addressed in a seamless and automated fashion.

Despite these promising results, certain limitations warrant discussion. The dataset, while rich and representative of real-world railway conditions, is constrained by the rarity of specific failure types, which may limit the generalizability of the models when deployed in different railway environments. Additionally, all experiments were conducted in a CPU-based environment without GPU acceleration; while sufficient for proof-of-concept, real-world deployment would benefit from further optimization and hardware acceleration to support large-scale, real-time monitoring.

In summary, this study reinforces the value of integrating machine learning and deep learning within a unified PdM framework, demonstrating measurable improvements in predictive accuracy and actionable insights for maintenance planning. Future research will aim to extend this framework with real-time deployment, incorporate transfer learning for rare fault types, and explore model interpretability to enhance decision support for maintenance engineers.

9. Conclusion

This paper introduced a multi-stage predictive maintenance system that combines machine learning and deep learning models to detect failures, classify fault types, and predict early degradation trends for railway engines. By leveraging sliding window segmentation, oversampling, and careful model tuning, the system demonstrated strong performance across binary classification, multi-class classification, and regression tasks.

Notably, oversampling significantly improved recall and F1-scores for both machine learning and deep learning models, effectively mitigating the challenges of class imbalance. Deep learning models (LSTM and GRU) excelled in modeling temporal dependencies and provided smooth, interpretable degradation trajectories, supporting predictive maintenance planning.

The system's modular design ensures flexibility and scalability, with potential applications across industrial sectors beyond the railway domain. Future work will focus on expanding the dataset with more diverse failure cases, incorporating explainability techniques to improve model transparency, and testing the system in real-time operational environments to validate its robustness and scalability.

References

- [1] Panagiotis Mallioris, Eirini Aivazidou, and Dimitrios Bechtsis. Predictive maintenance in industry 4.0: A systematic multi-sector mapping. *CIRP Journal of Manufacturing Science and Technology*, 50:80–103, 2024.
- [2] Zaharah Allah Bukhsh, A Saeed, and Irina Stipanovic. A machine learning approach for maintenance prediction of railway assets. *Transp. Res. Arena*, 2018, 2018.
- [3] João Antunes Rodrigues, José Torres Farinha, Mateus Mendes, Ricardo Mateus, and António Marques Cardoso. Short and long forecast to implement predictive maintenance in a pulp industry. *Eksplotacja i niezawodność*, 24(1), 2022.
- [4] Ali Hakami. Strategies for overcoming data scarcity, imbalance, and feature selection challenges in machine learning models for predictive maintenance. *Scientific Reports*, 14(1):9645, 2024.
- [5] Luis Rojas, Álvaro Peña, and José García. Ai-driven predictive maintenance in mining: A systematic literature review on fault detection, digital twins, and intelligent asset management. *Applied Sciences*, 15(6):3337, 2025.
- [6] Luigi De Simone, Enzo Caputo, Marcello Cinque, Antonio Galli, Vincenzo Moscato, Stefano Russo, Guido Cesaro, Vincenzo Criscuolo, and Giuseppe Giannini. Lstm-based failure prediction for railway rolling stock equipment. *Expert Systems with Applications*, 222:119767, 2023.
- [7] Pratik Vinayak Jadhav, Siddharth Sonkavade, Shivali Amit Wagle, Preksha Pareek, Ketan Kotecha, Tanupriya Choudhury, et al. A multi-task model for failure identification and gps assessment in metro trains. *AIMS Environmental Science*, 11(6), 2024.
- [8] Minh-Huong Le-Nguyen, Fabien Turgis, Pierre-Emmanuel Fayemi, and Albert Bifet. Real-time learning for real-time data: online machine learning for predictive maintenance of railway systems. *Transportation Research Procedia*, 72:171–178, 2023. TRA Lisbon 2022 Conference Proceedings Transport Research Arena (TRA Lisbon 2022), 14th–17th November 2022, Lisboa, Portugal.
- [9] Oscar Serradilla, Ekhi Zugasti, Julian Ramirez de Okariz, Jon Rodriguez, and Urko Zurutuza. Methodology for data-driven predictive maintenance models design, development and implementation on manufacturing guided by domain knowledge. *International Journal of Computer Integrated Manufacturing*, 35(12):1310–1334, 2022.
- [10] Zhe Li, Qian He, and Jingyue Li. A survey of deep learning-driven architecture for predictive maintenance. *Engineering applications of artificial intelligence*, 133:108285, 2024.
- [11] Jakub Wróbel, Paweł Bury, Mateusz Zajac, Artur Kierzkowski, Sławomir Tubek, and Jędrzej Blaut. Fault detection and diagnostic methods for railway systems—a literature survey. *Advances in Science and Technology. Research Journal*, 18(6), 2024.
- [12] Baotong Chen, Yan Liu, Chunhua Zhang, and Zhongren Wang. Time series data for equipment reliability analysis with deep learning. *Ieee Access*, 8:105484–105493, 2020.
- [13] Bruno Veloso, Rita P Ribeiro, João Gama, and Pedro Mota Pereira. The metropt dataset for predictive maintenance. *Scientific Data*, 9(1):764, 2022.
- [14] Mario Binder, Vitaliy Mezhyuev, and Martin Tschandl. Predictive maintenance for railway domain: A systematic literature review. *IEEE Engineering Management Review*, 51(2):120–140, 2023.
- [15] Qiankun Wang and Hongfang Lu. Machine learning methods for predicting residual strength in corroded oil and gas steel pipes. *npj Materials Degradation*, 9(1):30, 2025.
- [16] Yashashree Mahale, Shrikrishna Kolhar, and Anjali S More. A comprehensive review on artificial intelligence driven predictive maintenance in vehicles: technologies, challenges and future research directions. *Discover Applied Sciences*, 7(4):243, 2025.
- [17] Caxton Okoh, Rajkumar Roy, and Jorn Mehnen. Predictive maintenance modelling for through-life engineering services. *Procedia CIRP*, 59:196–201, 2017.
- [18] A.K.S. Jardine, D. Lin, and D. Banjevic. A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7):1483–1510, 2006.
- [19] X.S. Si, W. Wang, C.H. Hu, and D.H. Zhou. Remaining useful life estimation—a review on the statistical data driven approaches. *European Journal of Operational Research*, 213(1):1–14, 2011.
- [20] A. Saxena and K. Goebel. Turbofan engine degradation simulation data set. NASA Ames Prognostics Data Repository, 2008. <https://www.nasa.gov/content/prognostics-center-of-excellence-data-set-repository>.